



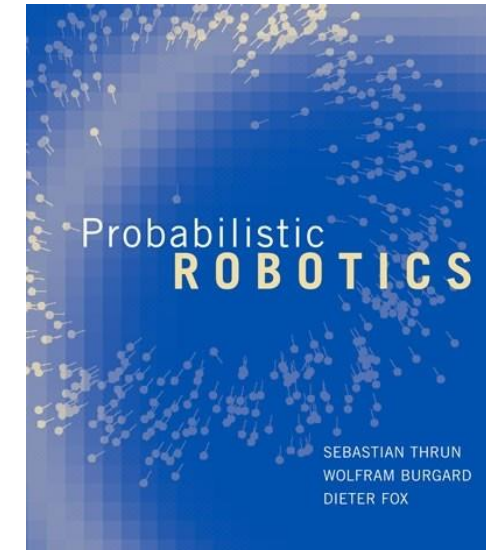
7 – Mapping and SLAM

Advanced Methods for Mapping and Self-localization in Robotics
MPC-MAP

Petr Gabrlik
Brno University of Technology
2026



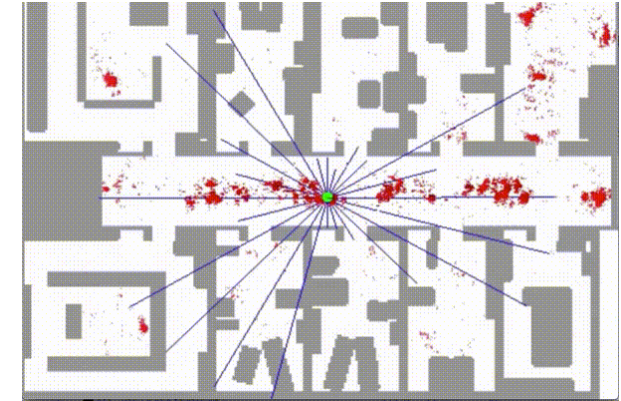
- The materials presented herein are mainly based on the ***Probabilistic Robotics*** book by Sebastian Thrun et al. [1]
- This presentation contains equations and graphics from the book; some images are adopted from the slides available at probabilistic-robotics.org.



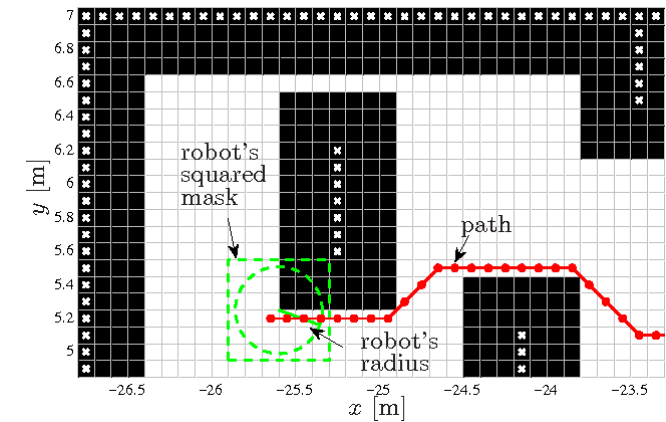
[1]



- Why are maps technically needed?
 - **Localization** – e.g. PF-based localization in known map
 - **Path Planning** – finding an effective path to the goal
 - **Obstacle Avoidance** – real-time map updates allow dynamic obstacle avoidance
 - Long-Term Autonomy, Multi-Robot Coordination etc.



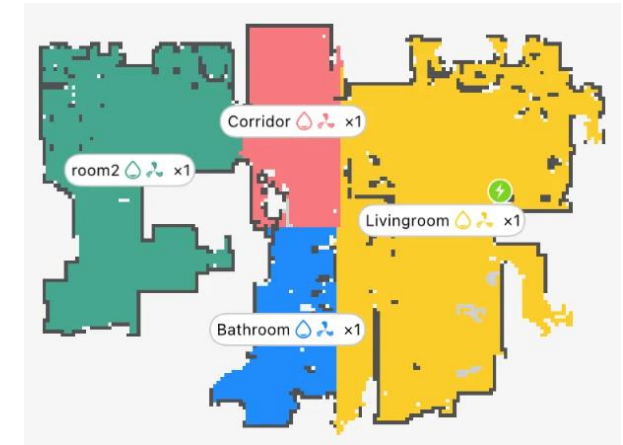
PF-based localization [1]



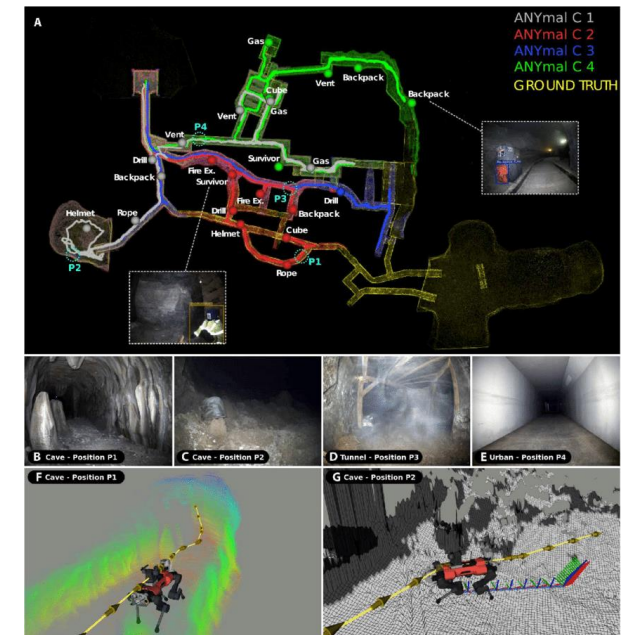
Path planning [2]



- Applications:
 - Service & Domestic Robots – vacuum cleaners use maps for efficient coverage
 - Warehouse Automation – maps are needed for an effective routing
 - Search & Rescue Operations – creating maps of unknown/altered environment
 - Etc.



A map created by a vacuum cleaner [1]



Search & Rescue, DARPA Sub-T challenge [2]

1. <https://rendeljkina.hu/en/reviews/home/robotic-vacuum-cleaning/tesvor-s7-pro-aes-robot-vacuum-test/>
2. https://www.researchgate.net/publication/360860225_CERBERUS_in_the_DARPA_Subterranean_Challenge

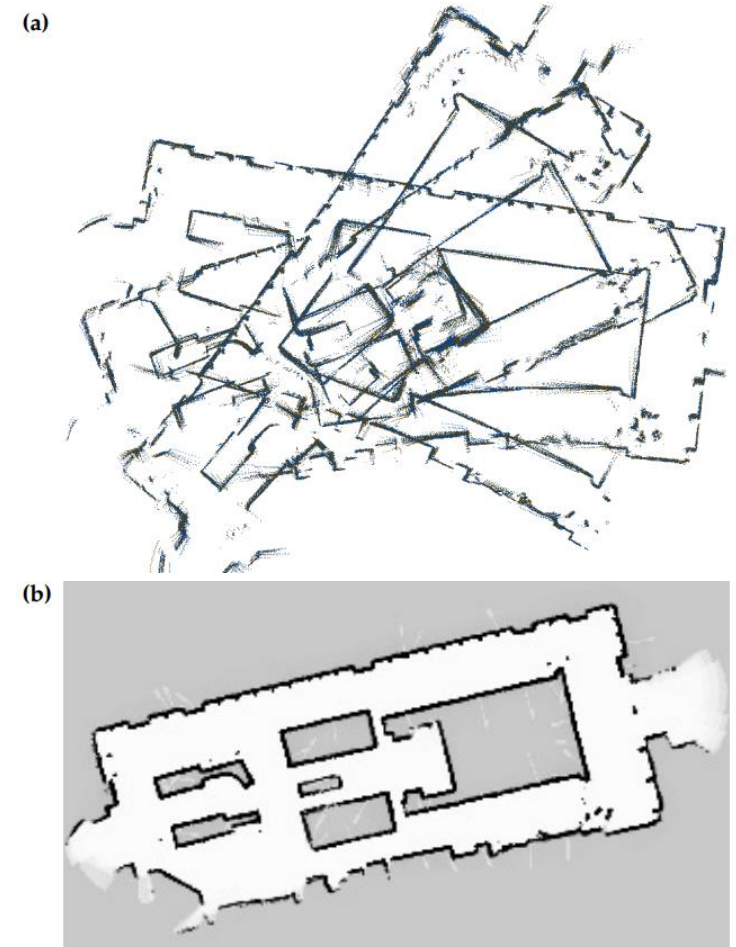


- **Naive approach**

- Use odometry for the localization task
- Use raw LiDAR data and known poses to draw a map

- **Problems**

- **Odometry is inaccurate** in long-term (integration error) → mapping requires precise pose estimate
- **Map** is represented by an **unordered set of points** → unsuitable format for path planning, grid-based representation is more suitable
- Map contains various **artefact** (e.g. from dynamic obstacles – people) → filtering is necessary – probabilistic approach (occupancy grid mapping)



[1] Raw range data, position indexed by odometry
(a). Occupancy grid map (b).

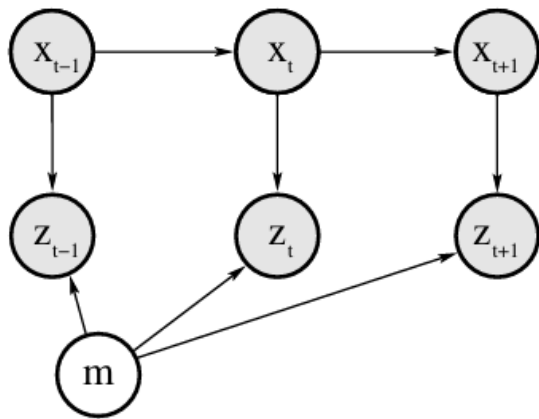


Mapping with known poses

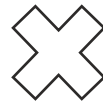
Occupancy Grid Mapping



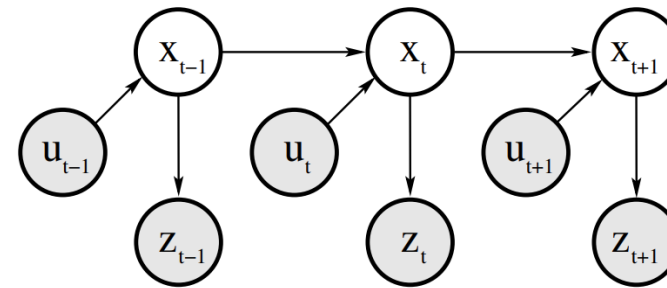
- **Mapping problem only – robot poses are known** (restrictive assumption).
- The poses are often acquired by means of a SLAM and the mapping is performed after that.



Mapping with known poses (shaded variables are known). Controls play no role. [1]



Compare mapping and localization



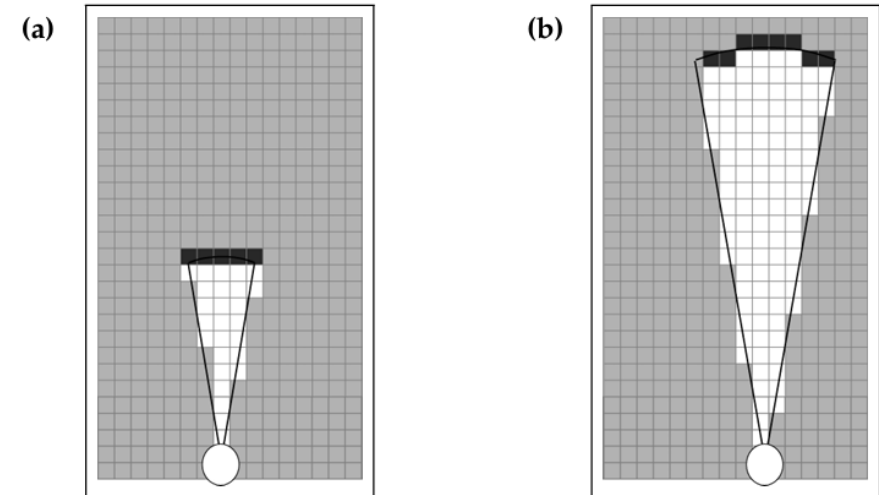
Bayes network – localization problem (shaded variables are known). [1]



- Occupancy Grid Mapping – a popular implementation of the Mapping With Known Poses
- *"Generating consistent maps from noisy and uncertain measurement data, under the assumption that the robot pose is known."* [1]
- The goal is to calculate the **posterior belief over maps** given data:
 - $p(m|z_{1:t}, x_{1:t})$
 - Control play no role since the path is already known.
 - Posterior belief represents **PDF over all possible maps** considering all measurements and path.



- Map representation
 - Field of random variables
 - Arranged in an evenly spaced grid
 - Each variable is binary (represents occupancy)
- Map m consists of cells $\mathbf{m}_i$: $m = \{\mathbf{m}_i\}$
- The posterior $p(m|z_{1:t}, x_{1:t})$ would lead to very high dimensionality
- In practice, approximated representation is used:
 - $p(\mathbf{m}_i|z_{1:t}, x_{1:t})$
- Estimation problem is now a binary problem – map is decomposed to independent cells



[1]

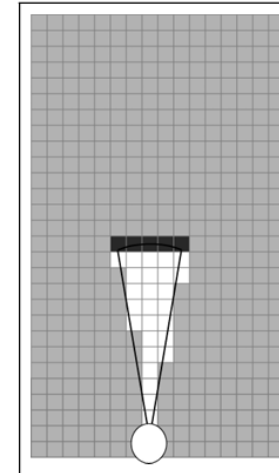


- This binary estimation problem is solvable with *binary Bayes filter* (see [1], Chapter 4.2).
- Occupancy grid mapping algorithm applies the *binary Bayes filter* to the *occupancy grid mapping problem*.
- Cells within the field of view are updated by the filter; the other ones remains unchanged.

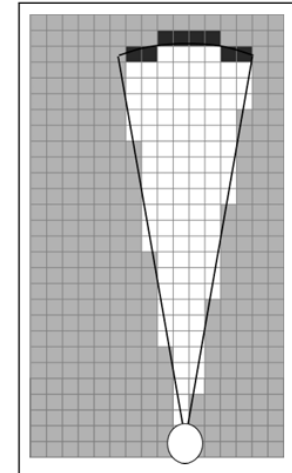
```
1:  Algorithm occupancy_grid_mapping( $\{l_{t-1,i}\}, x_t, z_t$ ):  
2:    for all cells  $m_i$  do  
3:      if  $m_i$  in perceptual field of  $z_t$  then  
4:         $l_{t,i} = l_{t-1,i} + \text{inverse\_sensor\_model}(m_i, x_t, z_t) - l_0$   
5:      else  
6:         $l_{t,i} = l_{t-1,i}$   
7:      endif  
8:    endfor  
9:    return  $\{l_{t,i}\}$ 
```

[1]

(a)



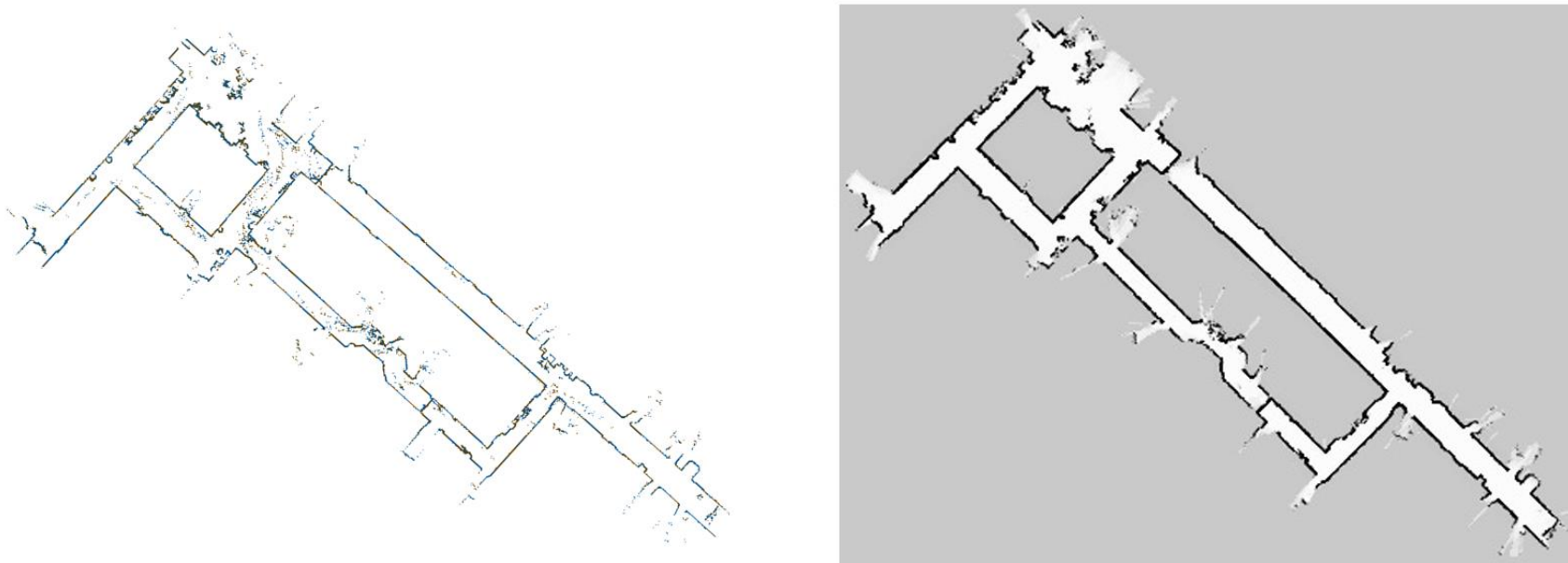
(b)



[1]



- Typical sensors utilized for the occupancy grid mapping are laser/ultrasound range finders.
- Essential features of the occupancy grid mapping:
 - Represents map in a suitable representation for path planning
 - Filters out “corrupted” data, e.g. dynamic obstacle (peoples), false reflections..



The comparison of the raw laser data with corrected poses from SLAM (left; comprises corrupted data, invalid obstacles) with occupancy grid built from the data (right; corrupted data are mostly filtered) [1]



SLAM

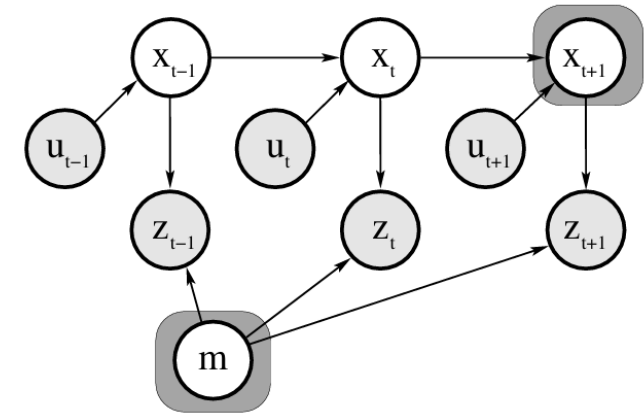
General aspect and overview



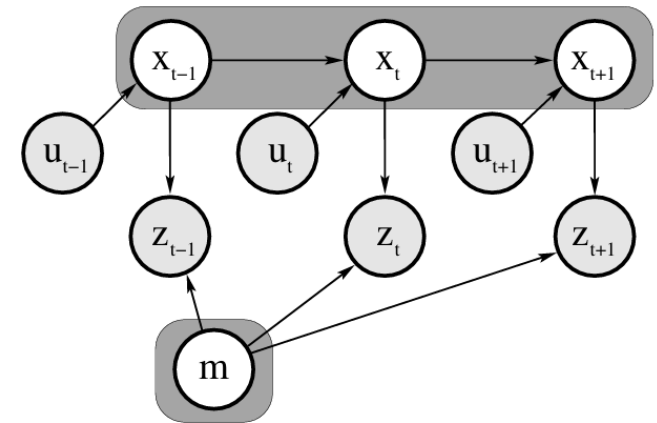
- **Fundamental problem in robotics – the map nor robot pose are not available.**
- Simultaneous Localization and Mapping (SLAM) – **general concept** to tackle the problem.
- Significantly more difficult than other problems in robotics.
 - Localization – estimating pose (in a known map)
 - Mapping – creating map while the pose is known
 - SLAM – **localizing robot** and **creating map** simultaneously



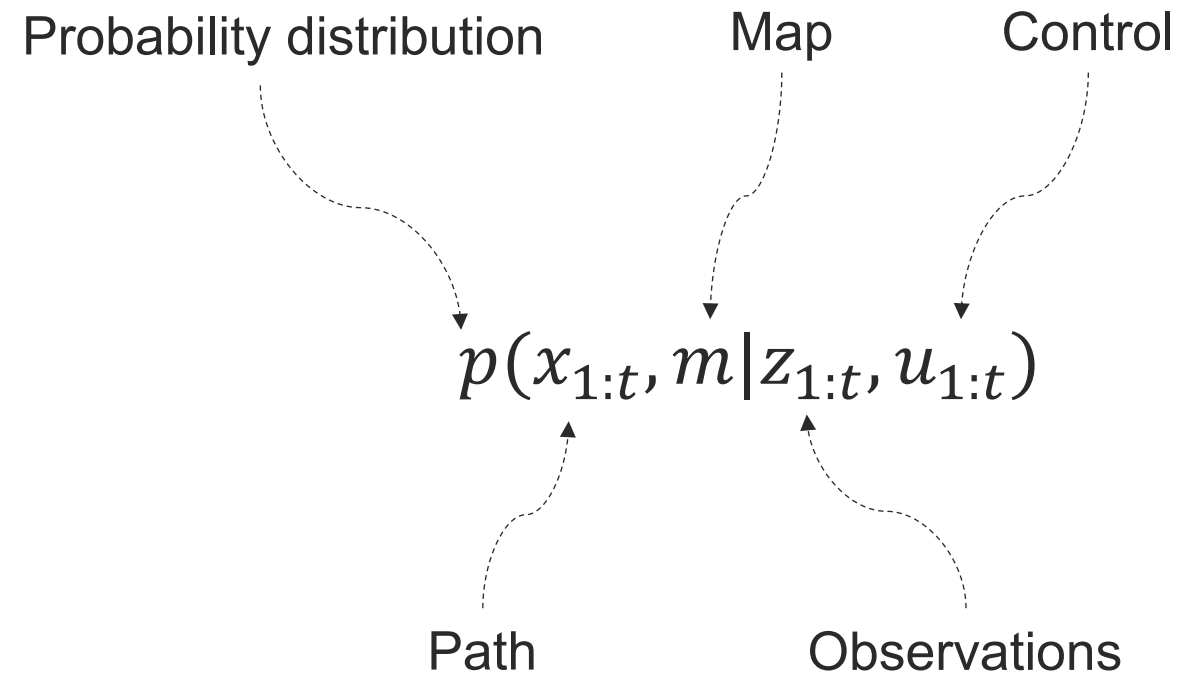
- From a probabilistic perspective, two forms exists:
- **Online SLAM**
 - Estimates posterior over the **momentary pose** x_t along with the map
 - $p(x_t, m | z_{1:t}, u_{1:t})$
 - Estimates only variables that persist at time t .
 - Past measurements and controls are typically discarded.
- **Full SLAM**
 - Estimates joint posterior over the **entire path** $x_{1:t}$ along with the map
 - $p(x_{1:t}, m | z_{1:t}, u_{1:t})$
 - Corrects past poses (e.g., loop closure corrections).



Online SLAM [1]

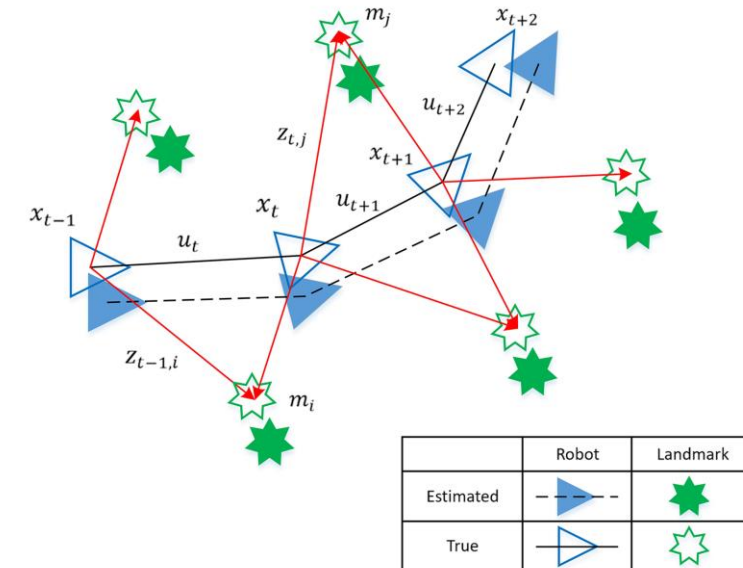


Full SLAM [1]

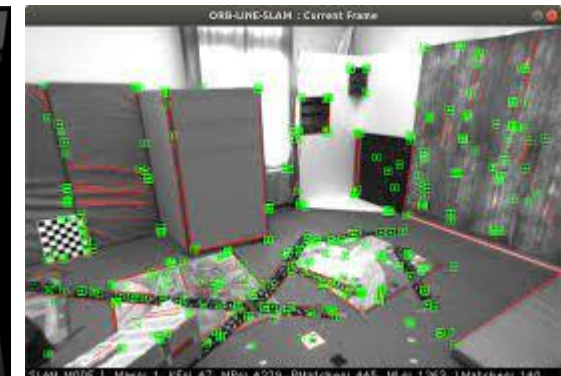




- Two-step mapping approach:
 - **SLAM** – estimates robot poses (path) and creates sparse environment map (represented by features or landmarks)
 - **Bayesian Occupancy Grid Mapping** – based on the known path, detailed occupancy grid map is created (using LiDAR/ultrasound rangefinder, camera)
- SLAM estimates positions of the objects in the map
 - Landmarks – feature-based representation
 - Object patches detected by range finders
 - ...
- Object association – was the object previously detected?



SLAM visualization [1]



Feature extraction [2, 3]

1. <https://www.mdpi.com/2079-9292/9/4/695>
2. https://www.researchgate.net/publication/338004888_Line-based_Visual_SLAM_by_Junction_Matching_and_Vanishing_Point_Alignment
3. ORB-LINE-SLAM: An Open-Source Stereo Visual SLAM System with Point and Line Features



Approach	SLAM type	Algorithms	Implementations
Filtering-based SLAM	Online	Bayes filter (EKF, PF)	EKF-SLAM, FastSLAM, UKF-SLAM
Optimization-based SLAM	Full, (online)	Graph-based optimization	Graph-SLAM, PGO, GTSAM,

SLAM types according processing method

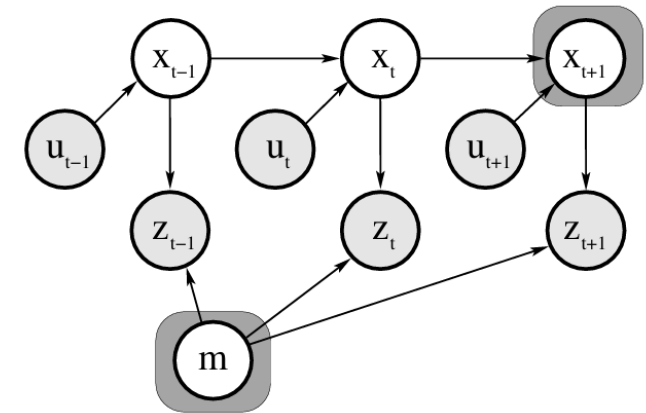
Category	Sensor	Implementations
V-SLAM	Camera (mono/stereo, RGB-D)	ORB-SLAM, LSD-SLAM
LiDAR-SLAM	LiDAR	LOAM, Google Cartographer, LeGO-LOAM
Multi-sensor SLAM	Camera + IMU + LiDAR	VINS-Mono, LIO-SAM, Kimera

SLAM types according sensors



■ Filtering-based SLAM

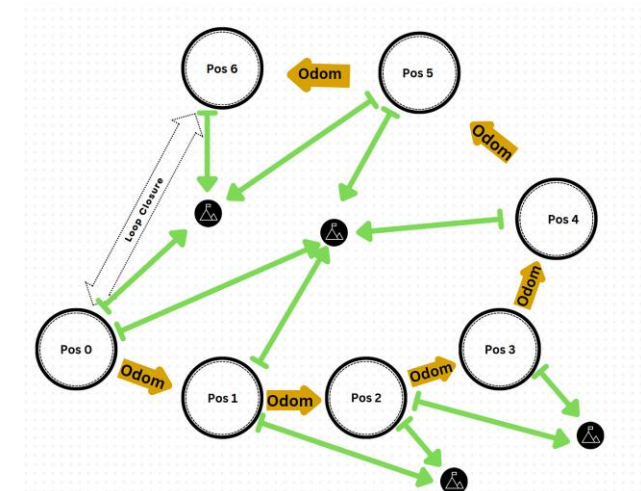
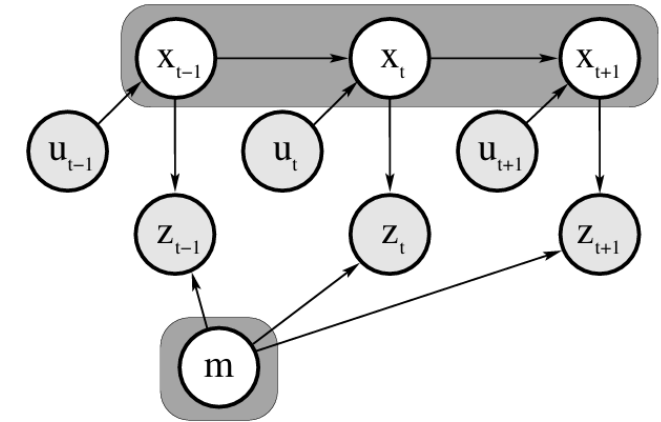
- Processes data sequentially – continuously updating the robot's state and map as new measurements arrive
- Online SLAM type
- Uses **Bayesian filtering** to estimate the probability distribution of the robot's state and landmarks
- Maintains **only the current estimate** of the trajectory and map
- Implementations:
 - EKF-SLAM - Gaussian approximation for state estimation
 - FastSLAM - Particle Filter for robot trajectory and EKF for landmark estimation
- Advantages:
 - Real-time, embedded applications, efficient for small maps
- Disadvantages:
 - Error accumulation, inefficient loop closure





■ Optimization-based SLAM

- **Graph Representation** - SLAM is modeled as a factor graph or pose graph
- Graph optimization task - the goal is to find the most probable robot trajectory and map by minimizing errors in sensor measurements (least squares..)
- Full SLAM type (may be online as well)
- Estimates (maintains) **entire path history**
- Implementations:
 - Graph-SLAM - pose graph solved using least squares optimization
 - PGO - Graph-SLAM that only optimizes robot poses (Google Cartographer, ORB-SLAM)
- Advantages:
 - Accurate, loop closure, large-scale maps
- Disadvantages:
 - Computational cost, memory intensive



Graph-based SLAM example [1]

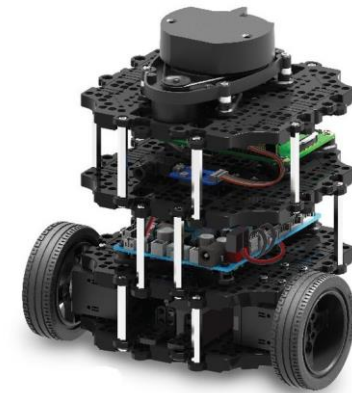


EKF-SLAM

Online SLAM implementation



- EKF-SLAM uses **EKF framework** (algorithm)
- Online SLAM – estimates actual robot pose and map
- Probabilistic approach - estimates the robot's pose and the positions of landmarks while maintaining uncertainty using a Gaussian distribution
- EKF-SLAM explained on the example comprising:
 - Differential-drive mobile robot
 - Range-angle sensor (landmark detector)





Non-linear model:

$$x_t = g(u_t, x_{t-1}) + \varepsilon_t$$

$$z_t = h(x_t) + \delta_t$$

defines the state transition probability $p(x_t|u_t, x_{t-1})$

defines the measurement probability $p(z_t|x_t)$

EKF algorithm:

$$1. \quad \bar{\mu}_t = g(u_t, \mu_{t-1})$$

$$2. \quad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$$

$$3. \quad K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$$

$$4. \quad \mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$$

$$5. \quad \Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$$

Prediction

$\overline{bel}(x_t)$

$bel(x_t)$

x_t, x_{t-1} $(n \times 1)$ state vectors

u_t $(m \times 1)$ control vector

g Nonlinear state transition function

ε_t $(n \times 1)$ process noise (state transition randomness); Gaussian with zero *mean* and *covariance* R_t $(n \times n)$

z_t $(k \times 1)$ measurement vector

h Nonlinear measurement function

δ_t $(k \times 1)$ measurement noise; Gaussian with zero *mean* and *covariance* Q_t $(k \times k)$



- **System state** is modeled as a vector comprising the **robot pose** and the **position of landmarks**
- $\mathbf{x} = \begin{bmatrix} \mathbf{x}_r \\ \mathbf{x}_l \end{bmatrix}$
- For a differential-drive robot, the state vector has the size of $(3 + 2n) \times 1$, n – number of landmarks

- $\mathbf{x} = \begin{bmatrix} x_r \\ y_r \\ \theta \\ x_1 \\ y_1 \\ \vdots \\ x_n \\ y_n \end{bmatrix}$



- The state uncertainty is represented by a **covariance matrix**
- $\Sigma = \begin{bmatrix} \Sigma_{rr} & \Sigma_{rl} \\ \Sigma_{lr} & \Sigma_{ll} \end{bmatrix}$
- For a differential-drive robot, the covariance matrix has the size of $(3 + 2n) \times (3 + 2n)$

- $\Sigma = \begin{bmatrix} \sigma_{x_r}^2 & \sigma_{y_r x_r} & \sigma_{y_r x_r} & & & \sigma_{y_n x_r} \\ \sigma_{x_r y_r} & \sigma_{y_r}^2 & & & & \\ & & \sigma_{\theta}^2 & & & \\ & & & \sigma_{x_1}^2 & & \\ & & & \sigma_{y_1}^2 & & \\ & & & \ddots & & \\ & & & & \sigma_{x_n}^2 & \\ \sigma_{x_r y_n} & & & & & \sigma_{y_n}^2 \end{bmatrix}$



- State transition function g comprises **robot motion** model and **landmark model**

$$\mathbf{x}^t = \mathbf{g}(\mathbf{u}^t, \mathbf{x}^{t-1}) + \boldsymbol{\varepsilon} = \begin{bmatrix} x_r^{t-1} + v \cos \theta^{t-1} \Delta t + \varepsilon_{x_r} \\ y_r^{t-1} + v \sin \theta^{t-1} \Delta t + \varepsilon_{y_r} \\ \theta_r^{t-1} + \omega \Delta t + \varepsilon_{\theta} \\ x_1^{t-1} \\ y_1^{t-1} \\ \vdots \\ x_n^{t-1} \\ y_n^{t-1} \end{bmatrix} = \mathbf{x}^{t-1} + \begin{bmatrix} v \cos \theta^{t-1} \Delta t \\ v \sin \theta^{t-1} \Delta t \\ \omega \Delta t \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} \varepsilon_{x_r} \\ \varepsilon_{y_r} \\ \varepsilon_{\theta} \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$



- **State transition matrix** is represented via the Jacobian matrix $\mathbf{G}$ of the function g
- $\mathbf{G}$ represents a linear approximation of the non-linear transition function g

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_r & 0 \\ 0 & \mathbf{I} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin \theta^{t-1} \Delta t & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & \cos \theta^{t-1} \Delta t & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \dots & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 1 \end{bmatrix}$$

- Robot model and landmark model are independent



- For the prediction, standard form of the EKF is utilized

$$\begin{aligned}\bar{\mu}_t &= g(u_t, \mu_{t-1}) \\ \bar{\Sigma}_t &= G_t \Sigma_{t-1} G_t^T + R_t\end{aligned}$$



- Sensor measures landmark's position in the polar coordinates $[r \ \phi]$ – distance and angle
- **Sensor model** (observation vector) has the size of $(2n) \times 1$; n – number of landmarks

$$\text{▪ } \mathbf{z}^t = \begin{bmatrix} r_1 \\ \phi_1 \\ \vdots \\ r_n \\ \phi_n \end{bmatrix} = \mathbf{h}(\mathbf{x}^t) + \boldsymbol{\delta} = \begin{bmatrix} \sqrt{(x_1 - x_r)^2 + (y_1 - y_r)^2} \\ \text{atan2}(y_1 - y_r, x_1 - x_r) - \theta \\ \vdots \\ \sqrt{(x_n - x_r)^2 + (y_n - y_r)^2} \\ \text{atan2}(y_n - y_r, x_n - x_r) - \theta \end{bmatrix} + \begin{bmatrix} \delta_r \\ \delta_\phi \\ \vdots \\ \delta_r \\ \delta_\phi \end{bmatrix}$$



- **Sensor model** is represented via the Jacobian matrix $\mathbf{H}$ of the function h
- $\mathbf{H}$ represents a linear approximation of the non-linear sensor model h
- $\mathbf{H} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}}$



- For the correction, standard form of the EKF is utilized

$$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$$

$$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$$



Petr Gabrlik

gabrlik@vut.cz

Brno University of Technology
Faculty of Electrical Engineering and Communication
Department of Control and Instrumentation



Robotics and AI Research Group